Experiment 4

Aim

To practically study the history, architecture, basic operations, and feature differences of three types of Version Control Systems using RCS (Local VCS), Subversion (Centralized VCS), and Mercurial (Distributed VCS).

PART A – Understanding Historical Evolution

Q1. Prepare a brief technical note (maximum 2 pages) explaining:

1. The evolution of Version Control Systems: Local, Centralized, and Distributed.
2. Architectural differences between the three models.
3. Limitations of each model that led to the development of the next.

Include architecture diagrams in your submission.

PART B – Local Version Control Using RCS

Task 1: Create a directory named 'VCS_Experiment'. Create a Python file 'app.py' with:

```
print('Version 1')
```

Task 2: Using RCS:

1. Place the file under version control.
2. Modify to Version 2 and commit.
3. Modify to Version 3 and commit.
4. Display revision history.
5. Retrieve Version 1.

Task 3: Analyze storage mechanism, collaboration capability, repository structure, and limitations.

PART C – Centralized Version Control Using Subversion (SVN)

Task 4: Create SVN repository with trunk, branches, and tags. Checkout trunk.

Task 5:

1. Add and commit Version 1.
2. Modify and commit Version 2 and Version 3.
3. View logs.
4. Create branch 'feature1'.
5. Commit changes in branch.
6. Merge branch back to trunk.

Task 6: Analyze server dependency, branching complexity, and history management.

PART D – Distributed Version Control Using Mercurial

Task 7:

1. Initialize repository.
2. Add and commit Version 1.
3. Modify and commit Version 2 and Version 3.

Task 8:

1. Create branch 'feature1'.
2. Commit feature changes.
3. View commit graph.
4. Clone repository.
5. Commit in clone.
6. Push changes to original repository.

Task 9: Analyze distributed architecture, offline capability, merge handling, and advantages.

PART E – Comparative Study

Task 10: Prepare a comparison table including:

- Architecture model
- Repository structure
- Collaboration support
- Offline capability
- Branching efficiency
- Merge handling
- Scalability
- Security considerations
- Suitability for academic research, enterprise systems, and open-source projects

PART F – Critical Thinking Questions

1. Why did distributed VCS become dominant in DevOps environments?
2. Compare conflict resolutions in SVN and Mercurial.
3. Which model is most suitable for CI/CD pipelines and why?
4. How does repository architecture affect distributed team performance?
5. If designing a new VCS today, what architectural improvements would you introduce?

Submission Instructions:

1. Students must upload a single PDF file containing the report of the experiment conducted.

2. The report should contain command history screenshots, revision logs, branching demonstration outputs, comparison table, analytical answers and any other details asked.

3. The PDF name should be in the form "StudentName_SAPID".